

数据库的完整性和数据库的安全性是两个既有联系又不尽相同的概念。

数据库的完整性是防止数据库中存在不符合语义的数据,也就是防止数据库中存在不正确的数据。数据的安全性是保护数据库防止恶意的破坏和非法的存取。因此,完整性检查和控制的防范对象是不合语义的、不正确的数据,防止它们进入数据库。安全性控制的防范对象是非法用户和非法操作,防止他们对数据库数据的非法存取。

为维护数据库的完整性,关系数据库管理系统必须能够实现以下功能。

1. 提供定义完整性约束的机制

完整性约束也称为完整性规则,是数据库中的数据必须满足的语义约束。它表达了给定的数据模型中数据及其联系所具有的制约和依存规则,用以限定符合数据模型的数据库状态以及状态的变化,以保证数据的正确、有效和相容。SQL 标准使用了一系列概念来描述完整性约束,包括关系模型的实体完整性、参照完整性和用户定义的完整性。这些完整性约束一般由 SQL 的数据定义语言语句来实现。它们作为数据库模式的一部分存入数据字典。

2. 提供检查完整性约束的方法

关系数据库管理系统中检查数据是否满足完整性约束的机制称为完整性检查。一般在 INSERT、UPDATE、DELETE 语句执行后开始检查,也可以在事务提交时检查。检查这些操作执行后数据库中的数据是否违背了完整性约束。

3. 提供完整性的违约处理方法

关系数据库管理系统若发现用户的操作违背了完整性约束,就采取一定的动作,如拒绝 (NO ACTION) 执行该操作或级联 (CASCADE) 执行其他操作,进行违约处理以保证数据的完整性。

早期的关系数据库管理系统不支持完整性约束检查,因为该项检查费时费资源。现在商用的关系数据库管理系统产品都支持完整性控制,不必由应用程序来完成,从而减轻了应用程序员的负担。

更重要的是,完整性控制已成为关系数据库管理系统核心支持的功能,从而能够为所有用户和应用提供一致的数据库完整性。由应用程序来实现完整性控制是有漏洞的,有的应用程序定义的完整性约束可能会被其他应用程序破坏,因此数据库数据的正确性仍然无法保障。

本书第2章2.3节已讲解了关系数据库三类完整性约束的基本概念,下面将介绍用 SQL 实现这些完整性控制功能的方法。

5.2 实体完整性

5.2.1 定义实体完整性

关系模型的实体完整性在 CREATE TABLE 中用 PRIMARY KEY 定义。对单属性构成的码有两种说明方法,一种是定义为列级约束,另一种是定义为表级约束。对多个属性构成的码只

有一种说明方法,即定义为表级约束。

[例 5.1] 创建“学生”表 Student, 将 Sno 属性定义为主码。

```
CREATE TABLE Student
    (Sno CHAR(8) PRIMARY KEY,          /* 在列级定义主码 */
     Sname CHAR(20) UNIQUE,
     Ssex CHAR(6),
     Sbirthdate Date,
     Smajor VARCHAR(40)
    );
```

或者

```
CREATE TABLE Student
    (Sno CHAR(8),
     Sname CHAR(20) UNIQUE,
     Ssex CHAR(6),
     Sbirthdate Date,
     Smajor VARCHAR(40),
     PRIMARY KEY(Sno)                  /* 在表级定义主码 */
    );
```

[例 5.2] 创建“学生选课”表 SC, 将(Sno,Cno)属性组定义为主码。

```
CREATE TABLE SC
    (Sno CHAR(8),
     Cno CHAR(5),
     Grade SMALLINT,
     Semester CHAR(5),                /* 开课学期 */
     Teachingclass CHAR(8),          /* 学生选修某一门课所在的教学班 */
     PRIMARY KEY (Sno, Cno)          /* 主码由两个属性构成,必须在表级定义主码 */
    );
```

5.2.2 实体完整性检查和违约处理

用 PRIMARY KEY 短语定义了关系的主码后,每当用户程序对基本表插入一条记录或对主码列进行更新操作时,关系数据库管理系统将按照实体完整性规则自动进行检查。包括:

- ① 检查主码值是否唯一,如果不唯一则拒绝插入或修改。
- ② 检查主码的各个属性是否为空,只要有一个为空就拒绝插入或修改。

从而保证了实体完整性。

检查记录中主码值是否唯一的一种方法是进行全表扫描,依次判断表中每一条记录的主码值与将插入记录的主码值(或修改的新主码值)是否相同,如图 5.1 所示。



图 5.1 用全表扫描方法检查主码唯一性

全表扫描是十分耗时的。为了避免对基本表进行全表扫描,关系数据库管理系统一般都在主码上自动建立一个索引,如图 5.2 所示的 $B+$ 树索引,通过索引查找基本表中是否已经存在新的主码值将大大提高效率。例如,如果新插入记录的主码值是 25,通过主码索引,从 $B+$ 树的根结点开始查找,只要读取三个结点就可以知道该主码值已经存在,即根结点(51)、中间结点(12 30)和叶结点(15 20 25)。所以不能插入这条记录。如果新插入记录的主码值是 86,也只要查找三个结点就可以知道该主码值不存在,所以可以插入该记录。

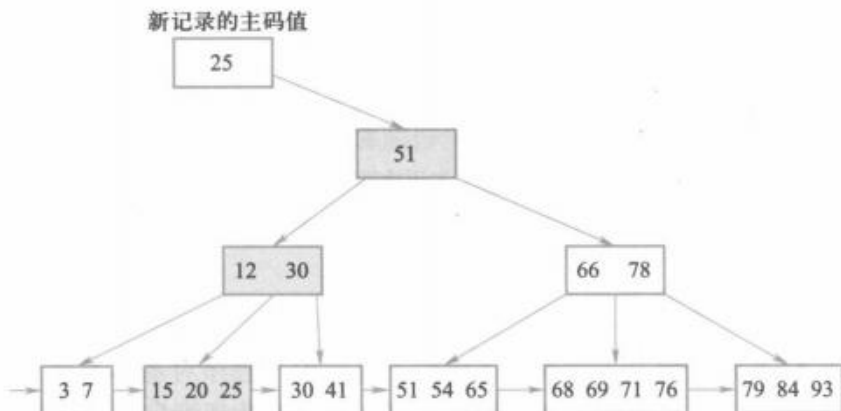


图 5.2 使用索引检查主码唯一性示例

5.3 参照完整性

5.3.1 定义参照完整性

关系模型的参照完整性在 CREATE TABLE 中用 FOREIGN KEY 短语定义哪些列为外码, 用 REFERENCES 短语指明这些外码参照哪些表的主码。

例如, 关系 SC 中一个元组表示一个学生选修某门课程的成绩, (Sno, Cno) 是主码。Sno、Cno 分别参照引用 Student 表的主码和 Course 表的主码。

[例 5.3] 定义 SC 中的参照完整性。

```
CREATE TABLE SC
(Sno CHAR(8),
 Cno CHAR(5),
 Grade SMALLINT,
 Semester CHAR(5),
 Teachingclass CHAR(8),
 PRIMARY KEY(Sno, Cno),      /* 在表级定义实体完整性 */
 FOREIGN KEY(Sno) REFERENCES Student(Sno),
                               /* 在表级定义参照完整性, Sno 是外码, 被参照表是 Student */
 FOREIGN KEY(Cno) REFERENCES Course(Cno)
                               /* 在表级定义参照完整性, Cno 是外码, 被参照表是 Course */
);
```

5.3.2 参照完整性检查和违约处理

参照完整性将两个表中的相应元组联系起来了。因此, 对被参照表和参照表进行更新(增、删、改)操作时有可能破坏参照完整性, 必须进行检查以保证这两个表的相容性。

例如, 对表 SC 和 Student 有 4 种可能破坏参照完整性的情况及违约处理, 如表 5.1 所示。

表 5.1 可能破坏参照完整性的情况及违约处理

被参照表 (如 Student)	参照表 (如 SC)	违约处理
可能破坏参照完整性	插入元组	拒绝
可能破坏参照完整性	修改外码值	拒绝
删除元组	可能破坏参照完整性	拒绝/级联删除/设置为空值
修改主码值	可能破坏参照完整性	拒绝/级联修改/设置为空值